

Merged BFS and DFS Program Using Switch Case and OpenMP

This PDF contains the merged C++ program for Parallel BFS and Parallel DFS using Switch Case and OpenMP.

```
// Merge BFS and DFS using Switch Case with OpenMP

#include <iostream>
#include <vector>
#include <queue>
#include <omp.h>

using namespace std;

class Graph {
    int V;
    vector<vector<int>>> adj;

public:
    Graph(int V) {
        this->V = V;
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // Parallel BFS
    void parallelBFS(int start) {
        vector<int> visited(V, 0);
        queue<int> q;

        visited[start] = 1;
        q.push(start);

        cout << "\\nParallel BFS Traversal: ";

        while (!q.empty()) {
            int size = q.size();

            #pragma omp parallel for
            for (int i = 0; i < size; i++) {

                int node = -1;

                #pragma omp critical
                {
                    if (!q.empty()) {
                        node = q.front();
                        q.pop();
                        cout << node << " ";
                    }
                }

                if (node == -1)
                    continue;

                for (int neighbor : adj[node]) {
                    if (!visited[neighbor]) {
                        #pragma omp critical
                        {
```

```

        if (!visited[neighbor]) {
            visited[neighbor] = 1;
            q.push(neighbor);
        }
    }
}

    }
}

    cout << endl;
}

// Parallel DFS
void parallelDFSUtil(int v, vector<bool> &visited) {

    visited[v] = true;

    #pragma omp critical
    cout << v << " ";

    #pragma omp parallel for
    for (int i = 0; i < adj[v].size(); i++) {

        int neighbor = adj[v][i];

        if (!visited[neighbor]) {

            #pragma omp task
            parallelDFSUtil(neighbor, visited);

        }
    }
}

void parallelDFS(int start) {

    vector<bool> visited(V, false);

    cout << "\\nParallel DFS Traversal: ";

    #pragma omp parallel
    {
        #pragma omp single
        {
            parallelDFSUtil(start, visited);
        }
    }

    cout << endl;
}

};

int main() {

    int V, E;

    cout << "Enter number of vertices: ";
    cin >> V;

    cout << "Enter number of edges: ";
    cin >> E;

    Graph g(V);

    cout << "Enter edges (u v):\\n";

    for (int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        g.addEdge(u, v);
    }
    int start;

```

```

cout << "Enter starting vertex: ";
cin >> start;

int choice;

cout << "\\n===== MENU =====\\n";
cout << "1. Parallel BFS\\n";
cout << "2. Parallel DFS\\n";
cout << "Enter your choice: ";
cin >> choice;

switch (choice) {
    case 1:
        g.parallelBFS(start);
        break;

    case 2:
        g.parallelDFS(start);
        break;

    default:
        cout << "Invalid Choice!" << endl;
}

return 0;
}

```